Free Monad

Free · KV-store program interpreted by Map vs logging · Build with Waqar

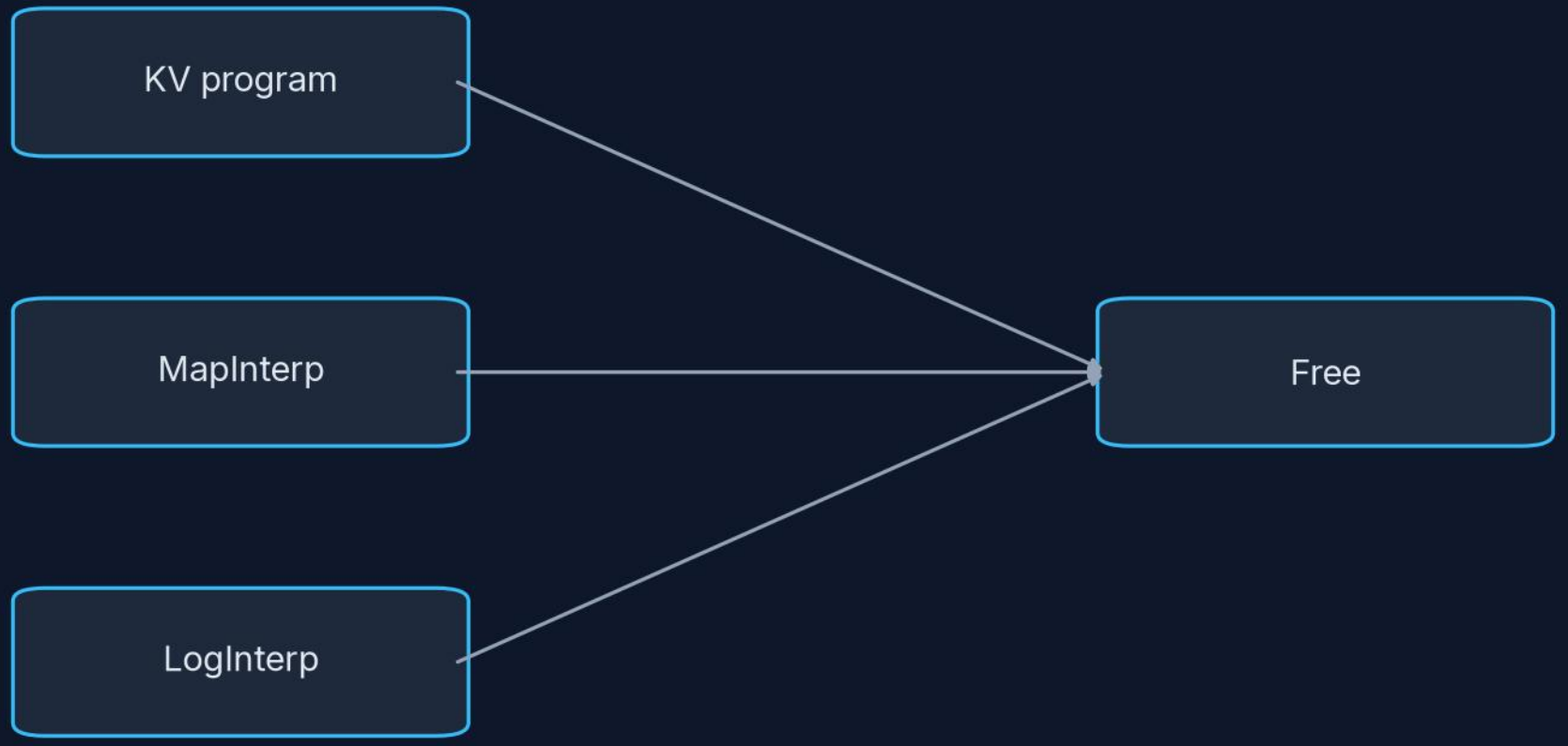
Today's Hook

- Problem: KV-store program interpreted by Map vs logging
- Pattern: Free — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- KV-store program interpreted by Map vs logging
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Free: KV-store program interpreted by Map vs logging

Real-World Scenario

- Running example: KV-store program interpreted by Map vs logging
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.functional.free;

import com.buildwithwaqar.patterns.functional.free.Free;
/** Problem: write a console script once; interpret with canned input in tests. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Free Monad problem: interpretable console script ===");
    Free<String> program = Free.printLine("name?").flatMap(x -> Free.readLine()).map(n -> "hi " + n);
    var out = new StringBuilder();
    System.out.println("result=" + Free.interpret(program, "Waqar", out));
    System.out.println("console=" + out.toString().trim());
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.free;

import java.util.function.Function;

/** Free monad over a console instruction set (simplified educational encoding). */
public abstract class Free<A> {
    private Free() {}

    public abstract <B> Free<B> flatMap(Function<? super A, Free<B>> f);

    public final <B> Free<B> map(Function<? super A, ? extends B> f) {
        return flatMap(a -> pure(f.apply(a)));
    }

    public static <A> Free<A> pure(A a) { return new Pure<>(a); }

    public static Free<Void> printLine(String s) { return new SuspendPrint(s); }

    public static Free<String> readLine() { return SuspendRead.INSTANCE; }

    private static final class Pure<A> extends Free<A> {
        private final A value;
        Pure(A value) { this.value = value; }
        A value() { return value; }
        public <B> Free<B> flatMap(Function<? super A, Free<B>> f) { return f.apply(value); }
    }
```

Solution (2)

```
private static final class SuspendPrint extends Free<Void> {
    private final String line;
    SuspendPrint(String line) { this.line = line; }
    public <B> Free<B> flatMap(Function<? super Void, Free<B>> f) {
        return new Bind<>(this, f);
    }
}

private static final class SuspendRead extends Free<String> {
    static final SuspendRead INSTANCE = new SuspendRead();
    public <B> Free<B> flatMap(Function<? super String, Free<B>> f) {
        return new Bind<>(this, f);
    }
}

private static final class Bind<A, B> extends Free<B> {
    private final Free<A> prev;
    private final Function<? super A, Free<B>> f;
    Bind(Free<A> prev, Function<? super A, Free<B>> f) {
        this.prev = prev;
        this.f = f;
    }
```

Solution (3)

```
        public <C> Free<C> flatMap(Function<? super B, Free<C>> g) {
            return new Bind<>(prev, a -> f.apply(a).flatMap(g));
        }
    }

    @SuppressWarnings({"rawtypes", "unchecked"})
    public static <A> A interpret(Free<A> program, String cannedInput, StringBuilder out) {
        Free cur = program;
        while (true) {
            if (cur instanceof Pure<?> p) {
                return (A) p.value();
            } else if (cur instanceof SuspendPrint sp) {
                out.append(sp.line).append('\n');
                cur = pure(null);
            } else if (cur instanceof SuspendRead) {
                cur = pure(cannedInput);
            } else if (cur instanceof Bind<?, ?> b) {
                Object inner = interpret(
```

Runnable main

```java
package com.buildwithwaqar.patterns.functional.free;

import com.buildwithwaqar.patterns.functional.free.Free;
/** Problem: write a console script once; interpret with canned input in tests. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Free Monad problem: interpretable console script ===");
    Free<String> program = Free.printLine("name?").flatMap(x -> Free.readLine()).map(n -> "hi " + n);
    var out = new StringBuilder();
    System.out.println("result=" + Free.interpret(program, "Waqar", out));
    System.out.println("console=" + out.toString().trim());
  }
}
```

Walkthrough Output

```
=== Free Monad problem: interpretable console script ===
result=hi Waqar
console=name?
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Free.

Related Patterns

- Contrast Free with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Free → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.